MEDs

Recordatorios de medicación, sin ruido

Universidad Politécnica de Chiapas

Documentación funcional y técnica

Guía de uso, arquitectura MVVM, API REST de CIMA, biometría, push y cobertura de la rúbrica del proyecto integrador.

INTEGRANTES

Ian Oscani Diaz Vazquez, Carlos Adrian Alarcon Valencia

Versión 26.1.1 - versionCode 4

9 de noviembre de 2026

com.meds.app

Contenido

Sección	Contenido
1.	Resumen ejecutivo y cambios de la versión 26.1.1
2.	Guía de uso: recorrido completo
3.	Arquitectura MVVM y organización del código
4.	Consumo de API y tratamiento de datos
5.	Interfaz con Jetpack Compose y navegación
6.	Gestión de datos y persistencia local
7.	Integraciones del dispositivo, biometría y push
8.	Estabilidad, validaciones y experiencia de uso
9.	Repositorio, documentación y versión entregable
10.	Demostración práctica y defensa técnica
11.	Cobertura de la rúbrica
12.	Evidencias y anexos

Este documento reemplaza a la versión 1.0. La versión 26.1.1 incorpora la consulta de medicamentos con la API REST de CIMA/AEMPS mediante Retrofit, reubica la biometría después del login y del onboarding, agrega el release firmado y actualiza las evidencias a 38 capturas.

1. Resumen ejecutivo

MEDs es una aplicación Android nativa para recordar medicamentos y sostener la constancia diaria. Está pensada para personas con TDAH y para quienes las acompañan: reduce la carga cognitiva, muestra solo lo necesario y convierte cada toma en una acción de un toque.

El proyecto modela dos roles, paciente y acompañante, un jardín que crece con la constancia, recordatorios locales, un widget de próxima toma, vinculación por QR y una consulta de medicamentos contra la API pública de CIMA/AEMPS. La versión 26.1.1 agrega Retrofit como cliente REST, una pantalla de consulta con resultados y ficha oficial, y un release firmado.

1.1 Problema y propuesta

- Olvidar una toma no es falta de voluntad: es carga cognitiva. Las apps genéricas añaden pasos, notificaciones ruidosas y pantallas densas.
- MEDs reduce la fricción: una toma se confirma con un toque; posponer cuesta otro toque; el estado del día se entiende de un vistazo.
- El modo acompañante resuelve el aislamiento: alguien de confianza ve el día y puede enviar un recordatorio, sin acceso al historial completo.
- La consulta CIMA permite verificar nombres, principios activos, presentaciones y documentos oficiales antes de guardar un medicamento.
- La constancia se refuerza con una planta y una racha, no con castigos ni culpa.

1.2 Ficha técnica

Aspecto	Detalle
Nombre y paquete	MEDs - com.meds.app
Propósito	Recordatorios de medicación, acompañamiento y consulta de medicamentos
Plataforma	Android 8.0 o superior (minSdk 26, targetSdk 37)
Lenguaje e interfaz	Kotlin, Jetpack Compose, Material 3
Arquitectura	MVVM + Repository, StateFlow y corrutinas; inyección manual
Backend	Firebase Authentication + Cloud Firestore (caché offline)
API REST	CIMA/AEMPS con Retrofit 2.11, Gson y OkHttp
Push	Firebase Cloud Messaging + Cloud Functions (Node 22)
Persistencia local	DataStore Preferences y DataStore de caché offline, SharedPreferences del widget y AlarmManager
Integraciones	QR con ZXing, biometría con androidx.biometric y widget AppWidgetProvider
Versión	26.1.1 - versionCode 4
Entrega	APK y AAB firmados en dist/

1.3 Cambios de la versión 26.1.1

- Consulta de medicamentos con la API REST pública de CIMA/AEMPS mediante Retrofit, con búsqueda, resultados, ficha, documentos y manejo de errores.
- El bloqueo con huella, rostro o PIN es opcional y ya no se pide antes del login: protege el espacio después de autenticarse y completar el onboarding.

- Tiempo de gracia de 5 minutos al volver del segundo plano, para no interrumpir cambios rápidos entre apps.
- Regeneración de QR que invalida el código anterior, con aviso y rechazo del código usado.
- Push de recordatorios y de vinculación con canales de alta importancia, sonido y vibración.
- Release 26.1.1 firmado, PDF actualizado y evidencias 32 a 38.
- Persistencia offline en DataStore: los medicamentos se guardan sin conexión, sobreviven al cierre de la app y se sincronizan con Firestore al recuperar la red.

2. Guía de uso: recorrido completo

2.1 Instalar y abrir

La variante de depuración se compila con .\gradlew.bat :app:assembleDebug y queda en app\build\outputs\apk\debug\app-debug.apk. El release firmado se compila con .\gradlew.bat :app:assembleRelease :app:bundleRelease y se guarda en dist\MEDs-release-26.1.1.apk y dist\MEDs-release-26.1.1.aab.

2.2 Cuenta y acceso

- Crear cuenta con correo y contraseña, con mínimo 6 caracteres.
- Iniciar sesión si la cuenta ya existe, con recuperación de contraseña por correo.
- Continuar sin cuenta: se crea una sesión anónima y se muestra una advertencia con los riesgos de no vincular un correo.
- La cuenta anónima se puede vincular a un correo desde Ajustes; la vinculación conserva el mismo uid y los datos.
- El registro y el login restauran el rol y el nombre desde Firestore.

2.3 Bloqueo con huella, rostro o PIN

- La app pregunta una sola vez si el usuario quiere bloquear MEDS con huella, rostro o el PIN del teléfono; la elección se puede cambiar en Ajustes.
- Si el usuario lo activa, se pide al entrar a un espacio ya creado, no en la pantalla de login.
- Si la huella falla o queda bloqueada, se puede usar el PIN, patrón o contraseña del teléfono.
- La pantalla de bloqueo permite cerrar sesión como salida de emergencia.
- Si el dispositivo no tiene biometría ni PIN configurado, la app no bloquea.
- Si la app vuelve del segundo plano después de 5 minutos, se pide de nuevo.
- Los cambios rápidos entre apps no vuelven a pedir desbloqueo.
- Al cerrar sesión se vuelve al login sin solicitud biométrica.

2.4 Paciente

- Hoy: próxima toma, estado del día, botones Ya la tomé y Posponer 10 min, historial y QR de acompañante.
- Medicamentos: lista, alta y edición; cada medicamento puede tener una o varias horas.
- Consultar medicamento: búsqueda en CIMA desde el editor, con resultados en español y ficha oficial.
- Jardín: planta, nivel, XP, racha y total de tomas.
- Ajustes: cuenta, vinculación de correo, cierre de sesión y versión.

2.5 Acompañante

- Escanear el QR del paciente o escribir el código de 8 caracteres.
- Ver en tiempo real las tomas del día, no el historial completo.
- Enviar un recordatorio breve y recibir la confirmación en la app.
- Desvincularse cuando quiera; el paciente también puede retirar el acceso.

- El código dura 10 minutos, es de un solo uso y el botón Generar nuevo QR invalida el anterior.

2.6 Widget, alarmas y push

- Widget: muestra la próxima toma pendiente; se actualiza cuando cambian las tomas y cada 30 minutos.
- Alarmas: notificaciones locales con acciones Ya la tomé y Posponer 10 min; se reprograman al reiniciar.
- Push: recordatorios y vinculación por FCM con canales de alta importancia; requiere desplegar Cloud Functions y el plan Blaze.
- El push con la app completamente cerrada se verificó en el diseño y en la carga de Functions; la prueba final depende del despliegue en Blaze.

3. Arquitectura MVVM y organización del código

La app separa interfaz, estado, contratos y acceso a datos. Ninguna pantalla habla directamente con Firebase ni con CIMA: las pantallas observan un StateFlow del ViewModel y envían eventos; el ViewModel depende de un contrato de domain/repository; la implementación en data/repository encapsula el SDK, Retrofit y las reglas de negocio.

Capa	Responsabilidad	Ejemplos
Interfaz (Compose)	Dibuja el estado y emite eventos; no accede a datos.	presentation/, core/ui/components, navigation/AppNavGraph
Estado (ViewModel)	Expone StateFlow, lanza corrutinas y recibe sus dependencias por constructor.	AppViewModel, PatientViewModel, MedicationInfoViewModel
Estado visible (UiState)	Contrato de datos de cada módulo: carga, contenido y error.	AppUiState, PatientUiState, CompanionUiState, AccountUiState
Composición (DI)	Crea una sola vez repositorios y servicios; entrega dependencias a los ViewModels.	AppContainer, MedsViewModelFactory
Datos (Repository)	Contratos en domain/repository e implementaciones en data/repository; batches, mapeo, reglas y Result.	AuthRepository, PatientRepository, CompanionRepository, MedicationInfoRepository
Fuentes	Identidad, base de datos, API REST, preferencias y alarmas.	FirebaseAuth, FirebaseFirestore, Retrofit, DataStore, AlarmManager

3.1 Paquetes principales

Paquete	Contenido
navigation	AppNavGraph, PatientNavGraph y Routes con destinos centralizados.
domain/model	Modelos inmutables de Firestore, CIMA, sesión y rol.
domain/repository	Contratos AuthRepository, UserRepository, PatientRepository, CompanionRepository y MedicationInfoRepository.
data/remote/api	CimaApi, CimaClient y configuración de Retrofit.
data/remote/dto	DTOs de la API de CIMA.
data/repository	Implementaciones *RepositoryImpl con Firestore, Firebase Auth, Retrofit y mapeos.
data/local	PreferencesStore y OfflineMedicationStore con DataStore: preferencias, caché de medicamentos y cola de cambios pendientes.
di	AppContainer con repositorios, PreferencesStore, OfflineMedicationStore y AlarmScheduler; MedsViewModelFactory.
integration/notifications	Alarmas, canales, receptores y MedsMessagingService.
presentation/patient	Hoy, medicamentos, consulta CIMA, jardín, historial y acompañante.
presentation/companion	Panel y vinculación del acompañante.
core/ui/components, integration/biometric	Carga, error y BiometricGate.
integration/widget, core/common	Widget de próxima toma, fechas, QR y códigos.

3.2 Estado, corrutinas y errores

Los ViewModels exponen un único StateFlow. Para combinar fuentes se usa combine; para cambiar de día o de paciente, flatMapLatest; para evitar trabajo innecesario, stateIn(WhileSubscribed(5_000)). Las pantallas recogen el estado con collectAsStateWithLifecycle. Las escrituras son suspend y devuelven Result; los errores se traducen a mensajes comprensibles y se muestran con snackbar o tarjetas de error.

3.3 Recorrido de la consulta CIMA

MedicationEditorScreen -> botón Consultar medicamento -> MedicationInfoScreen -> MedicationInfoViewModel.search -> MedicationInfoRepository.search -> CimaApi.medicamentos -> CIMA/AEMPS. El resultado se mapea a MedicationInfo y se muestra en tarjetas. Al elegir Usar este nombre o Usar principio activo, el nombre viaja por savedStateHandle al editor y se aplica al campo Nombre.

```
interface CimaApi {
    @GET("medicamentos")
    suspend fun search(
        @Query("nombre") name: String,
        @Query("pagina") page: Int = 1
    ): CimaSearchResponse

    @GET("medicamento")
    suspend fun detail(
        @Query("nregistro") registry: String
    ): CimaDetail
}
```

```
class MedicationInfoRepositoryImpl(
    private val api: CimaApi = CimaClient.api
) : MedicationInfoRepository {
    override suspend fun search(rawQuery: String): Result =
        runCatching {
            val query = rawQuery.trim()
            require(query.length >= 2) { "Escribe al menos 2 caracteres." }
            val response = api.search(query, page = 1)
            val sorted = response.resultados
                .map { it.toMedicationInfo() }
                .sortedWith(
                    compareByDescending {
                        it.activeSubstance.equals(query, ignoreCase = true)
                    }.thenByDescending { it.commercialized }
                     .thenBy { it.displayName.lowercase() }
                )
            MedicationSearchResult(sorted.take(20), response.totalFilas)
        }
}
```

3.4 Inyección de dependencias

La inyección es manual y explícita. MedsApp inicializa Firebase, crea un único AppContainer y le pasa el contenedor a la factoría MedsViewModelFactory. Los ViewModels reciben sus dependencias por constructor y ya no construyen repositorios: dependen solo de los contratos de domain/repository. La factoría obtiene el contenedor desde APPLICATION_KEY, que Compose y Navigation incluyen en las CreationExtras, y devuelve la instancia correcta para cada viewModel().

```
class AppContainer(context: Context) {
    private val auth = FirebaseAuth.getInstance()
    private val firestore = FirebaseFirestore.getInstance()

    val preferencesStore = PreferencesStore(context.applicationContext)
    val alarmScheduler = AlarmScheduler(context.applicationContext)
    val authRepository: AuthRepository = AuthRepositoryImpl(auth)
    val userRepository: UserRepository = UserRepositoryImpl(firestore)
    val patientRepository: PatientRepository =
        PatientRepositoryImpl(context, firestore, auth, alarmScheduler)
    val companionRepository: CompanionRepository =
        CompanionRepositoryImpl(firestore, auth)
    val medicationInfoRepository: MedicationInfoRepository =
        MedicationInfoRepositoryImpl()
```

```
}
```

```
val MedsViewModelFactory: ViewModelProvider.Factory = viewModelFactory {
    initializer {
        val container = container()
        PatientViewModel(
            repository = container.patientRepository,
            companionRepository = container.companionRepository,
            authRepository = container.authRepository
        )
    }
    // ... AppViewModel, PatientCompanionViewModel,
    // CompanionViewModel y MedicationInfoViewModel
}
```

Es una decisión deliberada: la app es pequeña, las dependencias son pocas y visibles, y no se introduce Hilt para no añadir complejidad. Las implementaciones aceptan FirebaseFirestore, FirebaseAuth, AlarmScheduler o CimaApi como parámetro, lo que permite sustituirlos en pruebas.

4. Consumo de API y tratamiento de datos

La app consume dos tipos de servicios externos: Firebase (Authentication, Cloud Firestore, Cloud Messaging y Cloud Functions) y la API REST pública de CIMA/AEMPS para la consulta de medicamentos. No hay datos simulados: las pantallas leen y escriben documentos reales del proyecto medsproto y consultas reales a CIMA.

4.1 API REST de CIMA/AEMPS

Elemento	Detalle
Base URL	https://cima.aemps.es/cima/rest/
Búsqueda	GET medicamentos?nombre={nombre}&pagina=1
Detalle	GET medicamento?nregistro={nregistro}
Cliente	Retrofit 2.11 + converter-gson + OkHttp
Autenticación	No requiere API key
Resultados	Se muestran 20, priorizando el principio activo exacto y los comercializados
Ficha	Principios activos, ATC, presentaciones, excipientes y enlaces a ficha técnica y prospecto

La búsqueda usa un User-Agent identificable (MEDs-Android/26.1.1), timeouts de 15 segundos de conexión y 25 de lectura. La respuesta de CIMA puede traer cientos de resultados; el repositorio toma los 20 más relevantes y conserva el total para mostrarlo en la interfaz. El detalle se consulta por nregistro cuando el usuario abre la ficha.

4.2 Tratamiento de errores

Situación	Respuesta en la app
Sin resultados	Estado Sin resultados con sugerencia de buscar por principio activo.
Sin conexión	Sin conexión. Revisa tu red e intenta de nuevo.
Timeout	La consulta tardó demasiado. Revisa tu conexión e intenta de nuevo.
HTTP 429	CIMA recibió demasiadas consultas. Espera un momento.
HTTP 5xx	El servicio de CIMA no está disponible. Intenta más tarde.
Respuesta con formato inesperado	La respuesta de CIMA no se pudo interpretar. Intenta de nuevo.

4.3 Operaciones de Firebase

Operación	Técnica	Dónde
Registro, login, vínculo y recuperación	FirebaseAuth (correo/contraseña y anónima)	AuthRepositoryImpl
Observar medicamentos, horarios, tomas y planta	Snapshots en tiempo real convertidos a Flow	PatientRepositoryImpl
Guardar medicamento y horario	Batch de dos documentos	PatientRepositoryImpl
Marcar toma	Batch: doseLog, planta y stock; cancela alarma	PatientRepositoryImpl
Invitar y vincular acompañante	Batch + validación de invitación en reglas	CompanionRepositoryImpl
Enviar y leer nudges	set / update con permisos por rol	CompanionRepositoryImpl
Registrar token FCM	Set(merge) en devices/{token}	UserRepositoryImpl

Operación	Técnica	Dónde
Perfil y rol	set(merge) en users/{uid}	UserRepositoryImpl

4.4 Seguridad de los datos

Las reglas de Firestore validan que cada usuario solo escriba sus propios documentos; el acompañante solo lee el día; las invitaciones son de un solo uso y expiran; los nudges solo los crea un acompañante vinculado; los tokens FCM solo los lee y escribe su dueño. El cliente nunca decide los permisos. La consulta CIMA es pública y no envía datos personales del usuario.

4.5 Qué resuelve el criterio 2

La rúbrica pide consumo de API y tratamiento de datos. La versión 1.0 se apoyaba solo en Firebase y dejaba un matiz sobre Retrofit. La versión 26.1.1 incorpora un cliente Retrofit real contra una API REST pública, con DTOs, mapeo a modelos de dominio, consulta de detalle, estados de carga y error, y documentación del flujo. El riesgo del criterio 2 queda cubierto.

5. Interfaz con Jetpack Compose y navegación

Toda la interfaz es Compose + Material 3. El tema Sereno clínico usa Manrope para texto y Fraunces para títulos, con paleta teal, fondos claros y modo oscuro. Los componentes reutilizables (MedsCard, PrimaryActionButton, SecondaryActionButton, TextActionButton, EmptyState) mantienen la consistencia.

5.1 Navegación

Ruta	Pantalla	Propósito
home	HomeScreen	Próxima toma, tomas del día e historial/QR.
medications	MedicationsScreen	Lista y edición.
medication_editor/new	MedicationEditorScreen	Alta de medicamento.
medication_editor/{id}	MedicationEditorScreen	Edición y consulta CIMA.
medication_info?query={query}	MedicationInfoScreen	Búsqueda y ficha CIMA.
garden	GardenScreen	Planta, nivel, XP y racha.
settings	AccountSettingsScreen	Cuenta, correo y cierre de sesión.
history	HistoryScreen	Últimos 7 días.
companions	PatientCompanionScreen	QR y personas vinculadas.

5.2 Retorno del nombre seleccionado

La pantalla de consulta devuelve el nombre mediante savedStateHandle del back stack del editor. El editor observa ese estado, aplica el nombre al campo y consume el valor. Así se evita acoplar la pantalla de consulta al ViewModel del paciente y se conserva el estado de navegación.

5.3 Accesibilidad y adaptación

- Contenido desplazable y layouts que se adaptan al ancho; probado en 1080x2400 y 1440x3120.
- Descripciones de contenido en iconos interactivos: ajustes, volver, borrar búsqueda, visibilidad de contraseña.
- Textos de error y ayudas junto a los campos; validación en el momento.
- Objetivos táctiles de al menos 48 dp en botones principales.
- Modo claro/oscuro según el sistema, con contraste cuidado.
- La consulta CIMA incluye aviso de que la información es de referencia y no sustituye el consejo médico.

6. Gestión de datos y persistencia local

Dato	Mecanismo	Supervivencia	Dónde
Rol, nombre y onboarding	DataStore Preferences	Reinicio de la app	PreferencesStore
Medicamentos y horarios	DataStore OfflineMedicationStore + Firestore	Reinicio y reconexión	PatientRepositoryImpl
Tomas, planta y vínculos	Cloud Firestore + caché offline	Reinicio y reconexión	Repositories
Próxima toma del widget	SharedPreferences	Reinicio y cambios de toma	NextDoseWidgetProvider
Alarmas locales	AlarmManager + SharedPreferences	Reinicio del dispositivo	AlarmScheduler / BootReceiver
Token FCM	Firestore users/{uid}/devices	Hasta cerrar sesión o invalidarse	UserRepository
Resultados CIMA	Memoria del ViewModel	Mientras vive la pantalla	MedicationInfoViewModel

6.1 DataStore como caché y cola de salida

La app usa dos almacenes DataStore independientes. meds_prefs conserva rol, nombre, onboarding y preferencias de biometría. meds_offline_cache conserva la última instantánea de medicamentos y horarios confirmada por Firestore, junto con una cola de altas, ediciones y borrados pendientes. La instantánea y la cola se serializan como JSON y se aíslan por owner_uid: al cambiar de usuario se limpian automáticamente.

Paso	Qué ocurre	Resultado sin red
1. Guardar	Se normaliza el borrador con UUID de cliente y se encola en DataStore antes de tocar la red.	El medicamento aparece de inmediato en la lista.
2. Escribir	PatientRepositoryImpl intenta el batch de Firestore con un límite de 6 segundos.	La escritura queda pendiente y se muestra el aviso de cambios sin sincronizar.
3. Confirmar	Si Firestore responde, el cambio sale de la cola y pasa a la instantánea local; las alarmas y las tomas del día se refrescan.	Nada se descarta: la cola se reintenta al abrir la app y cada 20 segundos.
4. Fusionar	Al leer, se combinan los datos remotos con la instantánea y la cola; los borrados pendientes filtran el medicamento remoto.	La interfaz nunca pierde un cambio local aunque Firestore aún no lo tenga.

```
suspend fun saveMedication(draft: MedicationDraft): Result = runCatching {
    val normalized = draft.copy(
        id = draft.id?.takeIf(String::isNotBlank) ?: UUID.randomUUID().toString(),
        scheduleId = draft.scheduleId?.takeIf(String::isNotBlank)
            ?: UUID.randomUUID().toString()
    )
    offlineStore.enqueueSave(uid(), normalized, isNew)
    scheduleAlarms(normalized)

    val outcome = runCatching {
        writeMedicationRemote(uid(), normalized, isNew, WRITE_TIMEOUT_MS)
    }
    if (outcome.isSuccess) {
        offlineStore.completeSave(uid(), normalized)
        refreshTodayLogs(uid(), normalized)
    } else if (!outcome.exceptionOrNull()!!.isOfflineError()) {
        offlineStore.discardSave(uid(), normalized.id.orEmpty())
        throw outcome.exceptionOrNull()!!
```

```
    }
}
```

La distinción entre error de red y error definitivo es importante: un timeout, IOException o UNAVAILABLE mantiene el cambio en la cola; un rechazo de permisos o de validación lo descarta para no bloquear la sincronización. Cada alta, edición o borrado usa el mismo identificador en local y en Firestore gracias al UUID generado en el cliente.

6.2 Por qué DataStore y no Room

El conjunto local es pequeño: medicamentos y horarios de un paciente, más una cola corta de cambios. DataStore cubre ese caso con lectura reactiva, escritura atómica y sin esquema ni migraciones. Room añadiría entidades, DAO, base de datos y migraciones para duplicar una fuente de verdad que sigue siendo Firestore; la caché local es solo una copia de lectura y una cola de salida, no una segunda base de datos. La consulta CIMA tampoco se persiste: es información de referencia y se consulta de nuevo cuando el usuario la necesita. Si en el futuro se requieren búsquedas locales sobre miles de registros, Room sería la opción natural.

6.3 Biometría y sesión

El estado de desbloqueo vive en la composición de BiometricGate. Se conserva durante la actividad, se pierde al reiniciar el proceso y se vuelve a pedir tras 5 minutos en segundo plano. No se guarda ningún dato biométrico en la app: el sensor y el sistema operativo gestionan la autenticación.

7. Integraciones del dispositivo, biometría y push

La rúbrica pide tres funcionalidades entre PDF, QR, cámara/galería, biometría, audio o widget. MEDs implementa QR (generar y escanear), biometría y widget. Además incluye alarmas, notificaciones locales y push con FCM como extras.

7.1 QR para vincular al acompañante

El paciente genera una invitación con QrCodeGenerator (ZXing) y la muestra como QR con el contenido meds://pair?code=XXXXXXXX. El acompañante puede escanearlo con ZXing Embedded o escribir el código de 8 caracteres. La invitación dura 10 minutos, es de un solo uso y las reglas validan que el paciente sea el dueño. Generar nuevo QR invalida el código anterior.

7.2 Bloqueo con huella, rostro o PIN

BiometricGate envuelve el contenido autenticado y con onboarding completo, no la pantalla de login. Usa BiometricPrompt con BIOMETRIC_WEAK y, según la versión de Android, permite el PIN del dispositivo como respaldo. Tras el onboarding, la app pregunta una sola vez si el usuario quiere proteger MEDS, y la preferencia se puede cambiar en Ajustes. Si la opción está activa, pide desbloqueo al entrar; si la huella falla o queda bloqueada, el usuario puede usar el PIN del teléfono o cerrar sesión desde la misma pantalla.

Un LifecycleEventObserver registra cuándo la app pasa a segundo plano. Al volver, si pasaron 5 minutos o más, se vuelve a bloquear; si fue un cambio rápido, no interrumpe. Es un equilibrio entre privacidad y la baja fricción que necesita una persona con TDAH.

```
private const val RELOCK_AFTER_MS = 5 * 60 * 1000L

Lifecycle.Event.ON_STOP -> {
    if (currentUnlocked && currentCanAuthenticate && !currentPrompting) {
        backgroundedAt = SystemClock.elapsedRealtime()
    }
}

Lifecycle.Event.ON_START -> {
    if (currentUnlocked && backgroundedAt > 0L &&
        SystemClock.elapsedRealtime() - backgroundedAt >= RELOCK_AFTER_MS
    ) {
        unlocked = false
    }
    backgroundedAt = 0L
}
```

7.3 Widget de próxima toma

NextDoseWidgetProvider es un AppWidgetProvider con RemoteViews. Guarda un snapshot de la próxima toma pendiente o pospuesta y lo pinta en el launcher. Se actualiza cuando cambian las tomas y cada 30 minutos (updatePeriodMillis=1800000). Si el día cambió, invita a abrir MEDs para refrescar.

7.4 Alarmas y notificaciones locales

AlarmScheduler programa alarmas exactas cuando el sistema lo permite y usa alarmas inexactas como respaldo. Las notificaciones incluyen las acciones Ya la tomé y Posponer 10 min, que ejecutan DoseActionReceiver. BootReceiver reprograma todo al reiniciar el dispositivo.

7.5 Push con FCM

La app registra el token FCM del paciente en users/{uid}/devices/{token} y lo elimina al cerrar sesión. Las Cloud Functions nudgeCreated y companionLinked escuchan Firestore y envían el push a todos los dispositivos del paciente. Los canales meds_nudges_v2 y meds_companions tienen importancia alta, sonido, vibración y aviso en pantalla.

Estado de entrega: la app y las Functions están implementadas y las Functions cargan en el emulador. Para que el push con la app cerrada funcione en producción hay que habilitar Blaze, ejecutar firebase login y desplegar firebase deploy --only functions,firestore:rules. Sin ese despliegue, la app sigue mostrando los recordatorios dentro de la interfaz, pero no como push del sistema. Android tampoco entrega push si el usuario usa Forzar detención.

7.6 Permisos declarados

Permiso	Uso
INTERNET	Firebase, CIMA y FCM.
POST_NOTIFICATIONS	Recordatorios locales y push (Android 13+).
SCHEDULE_EXACT_ALARM	Alarmas exactas; si no se concede, se usan inexactas.
RECEIVE_BOOT_COMPLETED	Reprogramar alarmas tras reiniciar.
CAMERA	Escaneo del QR con ZXing.
VIBRATE / WAKE_LOCK	Notificaciones y alarmas confiables.

8. Estabilidad, validaciones y experiencia de uso

8.1 Validaciones

Campo / flujo	Validación
Correo	Formato con expresión regular; error junto al campo.
Contraseña	Mínimo 6 caracteres; confirmación obligatoria en registro y vinculación.
Medicamento	Nombre y al menos una hora; dosis y forma con valores por defecto.
Código de acompañante	8 caracteres alfanuméricos; existencia, expiración y no uso previo.
Nudge	Mensaje obligatorio, máximo 160 caracteres.
Stock	Se descuenta solo si hay existencias; nunca queda negativo.
Búsqueda CIMA	Mínimo 2 caracteres; se recorta el texto y se limita a 80.

8.2 Errores y recuperación

Situación	Respuesta
Credenciales inválidas	Mensaje claro; no se cierra la app.
Correo ya registrado	Se sugiere iniciar sesión o usar otro correo.
Contraseña débil	Se indica el mínimo de 6 caracteres.
Sin conexión	Firestore usa caché; CIMA muestra error de red y permite reintentar.
Permiso retirado al acompañante	Mensaje específico y opción de desvincularse.
Perfil lento	Timeout de 8 s; la app no queda bloqueada.
Doble toque en una toma	Se comprueba el estado antes de volver a escribir.
ID numérico grande de CIMA	Se corrigió el modelo a Long tras una prueba real.
Vinculación recién creada	Reintento breve ante PERMISSION_DENIED transitorio.

8.3 Salvaguardas de experiencia

- Advertencia antes de continuar sin cuenta y confirmación al cerrar sesión.
- Estados de carga, vacío y error; snackbars para fallos de escritura.
- Cambio de día detectado cada 30 segundos; el widget avisa si quedó obsoleto.
- La confirmación de toma es idempotente y cancela la notificación correspondiente.
- El modo acompañante muestra solo el día y permite desvincularse.
- La biometría no interrumpe cambios rápidos de app; solo protege tras 5 minutos fuera.

8.4 Pruebas realizadas

Se verificaron los recorridos en emulador Pixel 6 Pro: login, registro, modo anónimo, advertencia, vinculación de correo, cierre de sesión, restauración de perfil, alta de medicamento, toma, historial, QR, biometría, widget, canales de notificación, consulta CIMA con resultados, ficha, sin resultados y error sin conexión.

Durante las pruebas se corrigieron defectos reales: la UI no se actualizaba al vincular el correo; la restauración de perfil podía reescribir preferencias del usuario anterior; el panel del acompañante mostraba un error falso justo después de vincular; y el ID vtm de CIMA desbordaba un Int y se cambió a Long. La versión 26.1.1 se probó en

emulador; se recomienda repetir la prueba física en el dispositivo antes de la entrega.

9. Repositorio, documentación y versión entregable

9.1 Estado actual

Elemento	Estado
Equipo	Ian Oscani Diaz Vazquez, Carlos Adrian Alarcon Valencia
Universidad	Universidad Politécnica de Chiapas
Fecha de entrega	9 de noviembre de 2026
Código fuente	Proyecto Android completo en Kotlin/Compose.
Repositorio	https://github.com/HVT-IAN/proyecto_movil-Med
Ramas	main, develop y feature/consulta-cima-biometria integrada a develop.
README	Actualizado con stack, API CIMA, biometría, push, release y estructura.
Documentación	Este PDF, versión 26.1.1.
Evidencias	38 capturas en evidencias/.
Versión	versionName 26.1.1, versionCode 4.
Release	APK y AAB firmados en dist/.
GitHub Release	v1.1 publicado con APK y AAB adjuntos; pendiente de publicar v26.1.1.
Firma	Keystore local no versionado; contraseña en keystore.properties, ignorado por Git.

9.2 Release firmado

El release se compila con assembleRelease y bundleRelease. El APK firmado queda en dist/MEDs-release-26.1.1.apk y el AAB en dist/MEDs-release-26.1.1.aab. La firma se verificó con apksigner; el certificado es CN=MEDs, OU=Proyecto integrador, O=MEDs, L=Madrid, ST=Madrid, C=ES. El Release v1.1 sigue publicado en GitHub con los artefactos anteriores; la versión 26.1.1 se publicará como un release nuevo.

El keystore y keystore.properties no se suben al repositorio. Hay que respaldarlos: sin ellos no se pueden publicar actualizaciones con la misma identidad. El APK de depuración también queda en dist/MEDs-debug-production.apk para pruebas internas.

9.3 Checklist antes de entregar

- Reglas de Firestore publicadas y probadas en producción.
- Release 26.1.1 firmado e instalado en el dispositivo físico.
- PDF y README actualizados.
- Evidencias 32 a 38 incluidas.
- Tag v26.1.1 y Release de GitHub publicados con APK y AAB adjuntos.
- Cuenta de demostración con correo y contraseña disponibles por canal aparte.
- Guion de 10 minutos ensayado.
- Push: desplegar Functions y Blaze si se va a demostrar con la app cerrada.

10. Demostración práctica y defensa técnica

10.1 Guion de 10 minutos

Minuto	Qué mostrar
0:00 - 1:00	Problema y propuesta: adherencia, TDAH, dos roles y promesa de bajo ruido.
1:00 - 2:30	Login o registro; advertencia anónima; vincular correo en Ajustes y mostrar que los datos se conservan. La biometría aparece después del onboarding, no antes.
2:30 - 4:00	Onboarding, alta de medicamento y consulta CIMA: buscar ibuprofeno, abrir ficha y usar el principio activo.
4:00 - 5:30	Pantalla Hoy, marcar una toma, posponer otra, historial y jardín.
5:30 - 7:30	Modo acompañante en un segundo dispositivo: generar QR, mostrar que el anterior queda inválido, escanear, ver el día y enviar un nudge.
7:30 - 8:30	Widget, notificación local con acciones, canales de push y desbloqueo biométrico.
8:30 - 10:00	Arquitectura: Repository, ViewModel, UI, Retrofit/CIMA, reglas y persistencia. Cerrar con la rúbrica.

10.2 Preguntas probables

Pregunta	Respuesta corta
¿Dónde está la llamada a la API?	En MedicationInfoRepository, con Retrofit contra CIMA; las pantallas nunca llaman a la API directamente.
¿Por qué CIMA y no openFDA?	CIMA es la API oficial española, devuelve datos en español y encaja con la app; openFDA quedó como alternativa documentada.
¿Por qué no Room?	Firestore es la fuente de verdad y ya tiene caché offline; DataStore cubre preferencias. La consulta CIMA es información de referencia y no se persiste.
¿Cómo funciona sin conexión?	Firestore conserva la caché; CIMA muestra un error de red claro y permite reintentar.
¿Cómo se protege la privacidad?	Reglas de Firestore, invitaciones de un solo uso y bloqueo opcional con huella, rostro o PIN.
¿Cuándo se pide el desbloqueo?	La app pregunta una sola vez después del onboarding si quieres bloquearla, y se puede cambiar en Ajustes. Si está activo, se pide al entrar y tras 5 minutos en segundo plano; si la huella falla, se usa el PIN del teléfono.
¿Qué pasa al cerrar sesión?	Se limpian las preferencias; una cuenta con correo se recupera al iniciar sesión; una anónima sin vincular se pierde y la UI lo advierte.
¿Qué falta?	Desplegar Functions/Blaze para el push real y probar el release en el dispositivo físico.

10.3 Mapa de código para la defensa

Tema	Archivos
Entrada	MainActivity.kt, MedsApp.kt
Autenticación	AuthRepository.kt (contrato), AuthRepositoryImpl.kt, AppViewModel.kt, AuthScreen.kt
API CIMA	CimaApi.kt, CimaModels.kt, MedicationInfoRepository.kt, MedicationInfoRepositoryImpl.kt, MedicationInfoViewModel.kt
Consulta CIMA UI	MedicationInfoScreen.kt, MedicationEditorScreen.kt, PatientApp.kt
Estado del paciente	PatientViewModel.kt
Inyección de dependencias	di/AppContainer.kt, di/MedsViewModelFactory.kt, MedsApp.kt
Firestore	PatientRepositoryImpl.kt, CompanionRepositoryImpl.kt, FirestoreMappers.kt

Tema	Archivos
Biometría	integration/biometric/BiometricGate.kt, navigation/AppNavGraph.kt
Push	MedsMessagingService.kt, NotificationHelper.kt, functions/index.js
Persistencia	PreferencesStore.kt, NextDoseWidgetProvider.kt, AlarmScheduler.kt
Seguridad	firestore.rules

11. Cobertura de la rúbrica

La rúbrica califica sobre 100 puntos y aporta el 60 % de la unidad. La tabla resume dónde se cubre cada criterio en la versión 26.1.1.

Criterio	Máx.	Evidencia en este documento	Estado
1. Arquitectura MVVM y organización	20	Secciones 3 y 10.3: capas, StateFlow, repositorios, DI manual y recorridos.	Cubierto
2. Consumo de API y tratamiento de datos	15	Sección 4: Firebase + Retrofit contra CIMA, DTOs, mapeo, errores y seguridad.	Cubierto
3. Interfaz Compose y navegación	15	Sección 5: NavHost, ruta medication_info, estados, tema y componentes.	Cubierto
4. Gestión de datos y persistencia	10	Sección 6: DataStore de preferencias y caché offline, cola de sincronización, Firestore, SharedPreferences y alarmas.	Cubierto
5. Integraciones del dispositivo	15	Sección 7: QR, biometría y widget; alarmas y push como extras.	Cubierto
6. Estabilidad, validaciones y UX	10	Sección 8: validaciones, errores, estados, pruebas y correcciones.	Cubierto
7. Repositorio, documentación y versión	5	Sección 9: repo, ramas, release firmado, PDF y 38 evidencias.	Cubierto
8. Demostración y defensa	10	Sección 10: guion de 10 minutos, mapa de código y preguntas.	Cubierto
Total	100	Criterios 1 a 8 cubiertos en la versión 26.1.1.	100/100

11.1 Riesgos y acciones

Riesgo	Nivel	Acción
CIMA no disponible o sin red	Medio	La app muestra error y reintento; la consulta no bloquea el resto de la app.
Guardado sin red antes de sincronizar	Bajo	La cola local conserva el cambio y lo reintenta; si se borran los datos de la app antes de sincronizar, ese cambio se pierde.
Push sin desplegar	Medio	Habilitar Blaze, firebase login y desplegar Functions/reglas antes de demostrar el push cerrado.
Prueba física pendiente	Medio	Instalar el release 26.1.1 en el dispositivo y repetir login, biometría, QR, widget y CIMA.
Keystore no respaldado	Bajo	Guardar keystore y keystore.properties en un lugar seguro, fuera de Git.
Cuenta anónima sin vincular	Bajo	Vincular un correo en la demo.

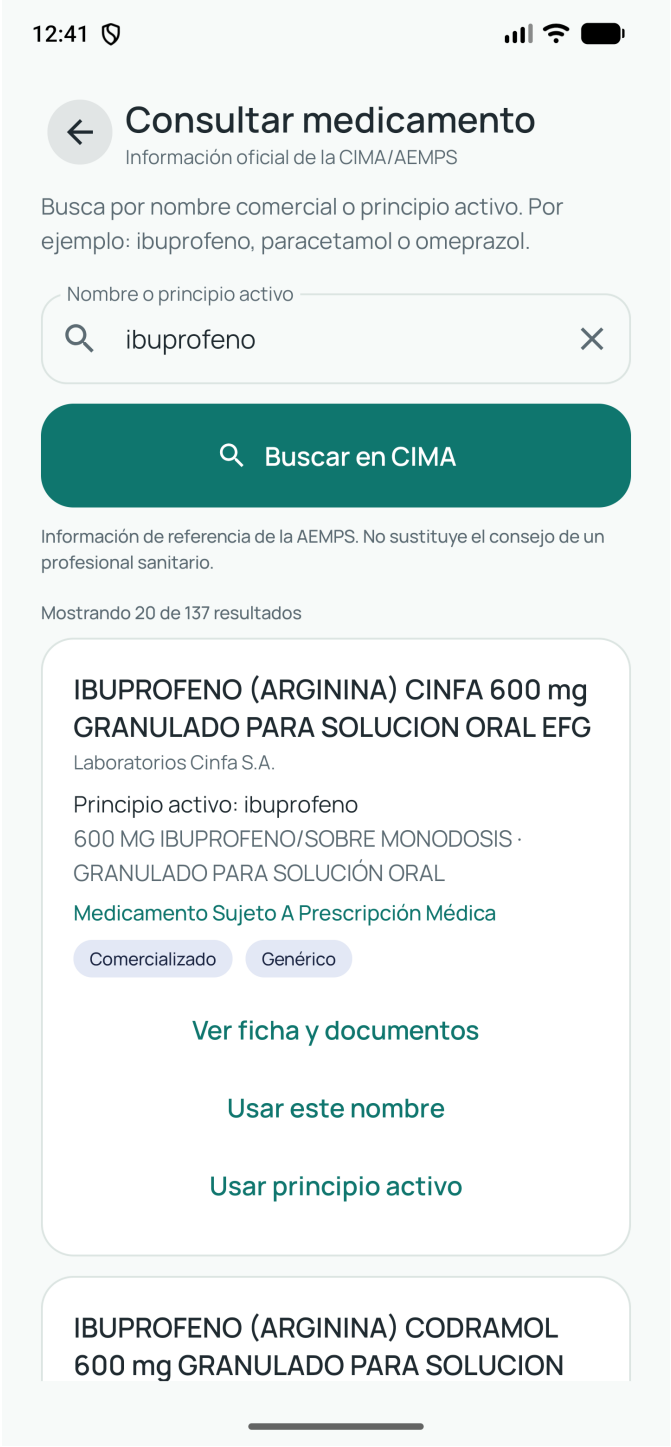
11.2 Conclusión

La versión 26.1.1 cubre los ocho criterios de la rúbrica. El criterio 2 deja de ser un matiz porque incorpora Retrofit y una API REST real; el criterio 7 se completa con repositorio, ramas, release firmado, documentación y evidencias. El push con la app cerrada queda como una funcionalidad extra cuyo despliegue en Blaze debe completarse antes de la demostración.

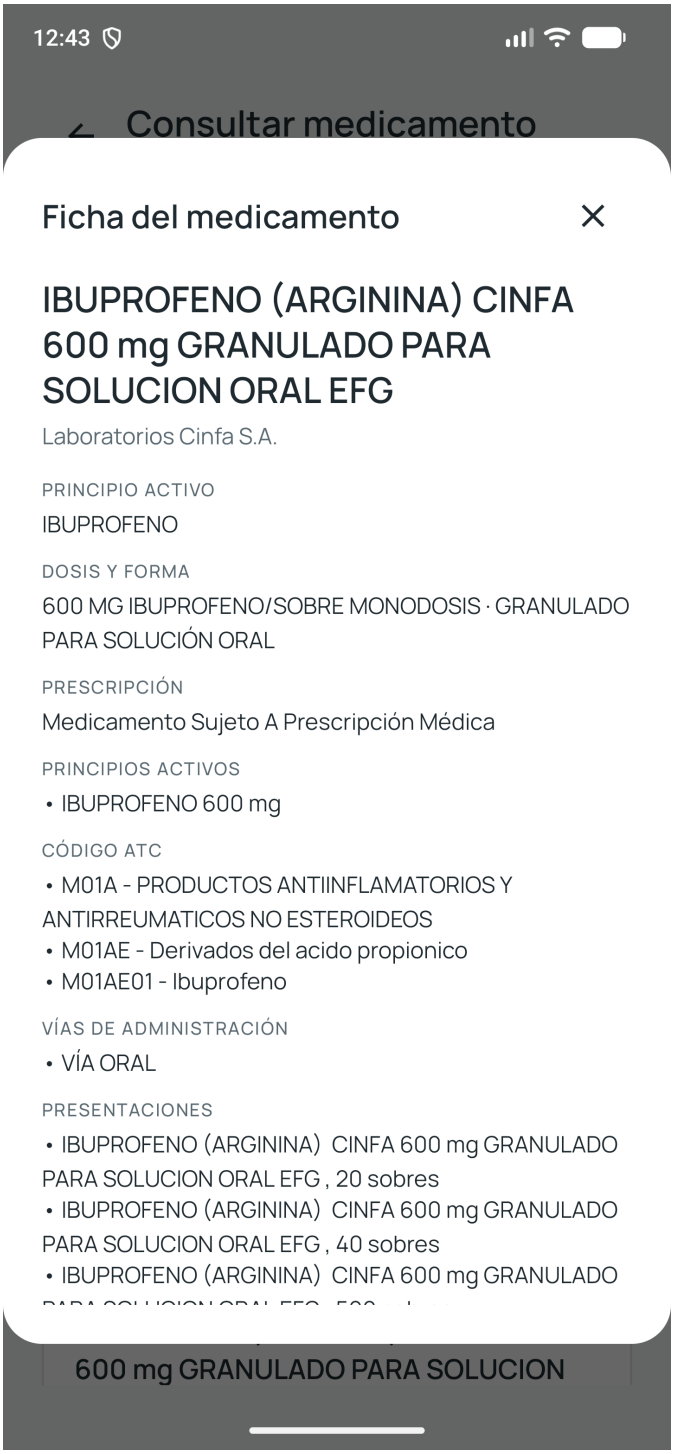
12. Evidencias y anexos

Las capturas están en la carpeta evidencias/. Las imágenes 32 a 38 documentan la consulta CIMA y la ubicación correcta de la biometría. Las capturas 21 a 31 documentan QR, vinculación, recordatorios y canales de push.

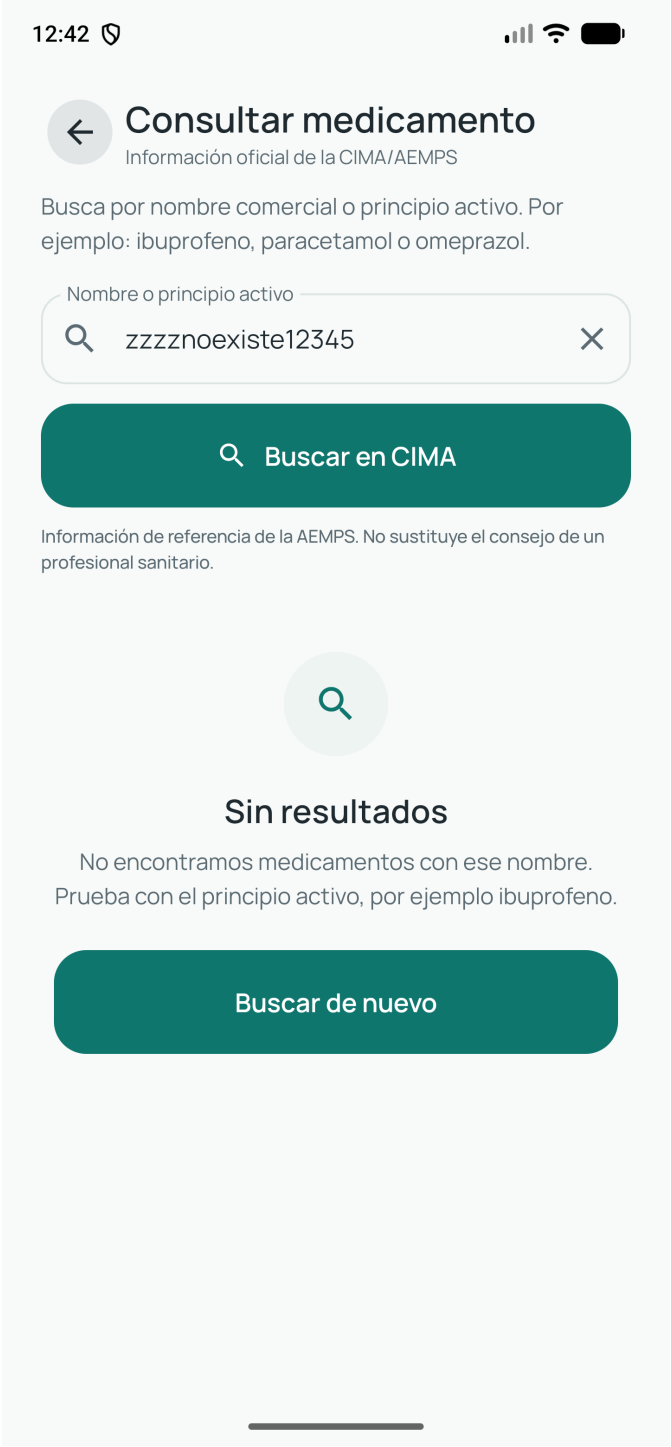
Archivo	Contenido
32_consulta_medicamento.png	Pantalla de consulta de medicamentos con CIMA/AEMPS.
33_resultados_cima.png	Resultados en español; 20 de 137, priorizando el principio activo.
34_detalle_medicamento.png	Ficha con principios activos, ATC, presentaciones y documentos.
35_sin_resultados.png	Estado sin resultados.
36_error_conexion.png	Error sin conexión con opción de reintentar.
37_biometria_tras_login.png	Bloqueo biométrico después de entrar a un espacio creado.
38_login_sin_biometria.png	Login sin solicitud de huella después de cerrar sesión.



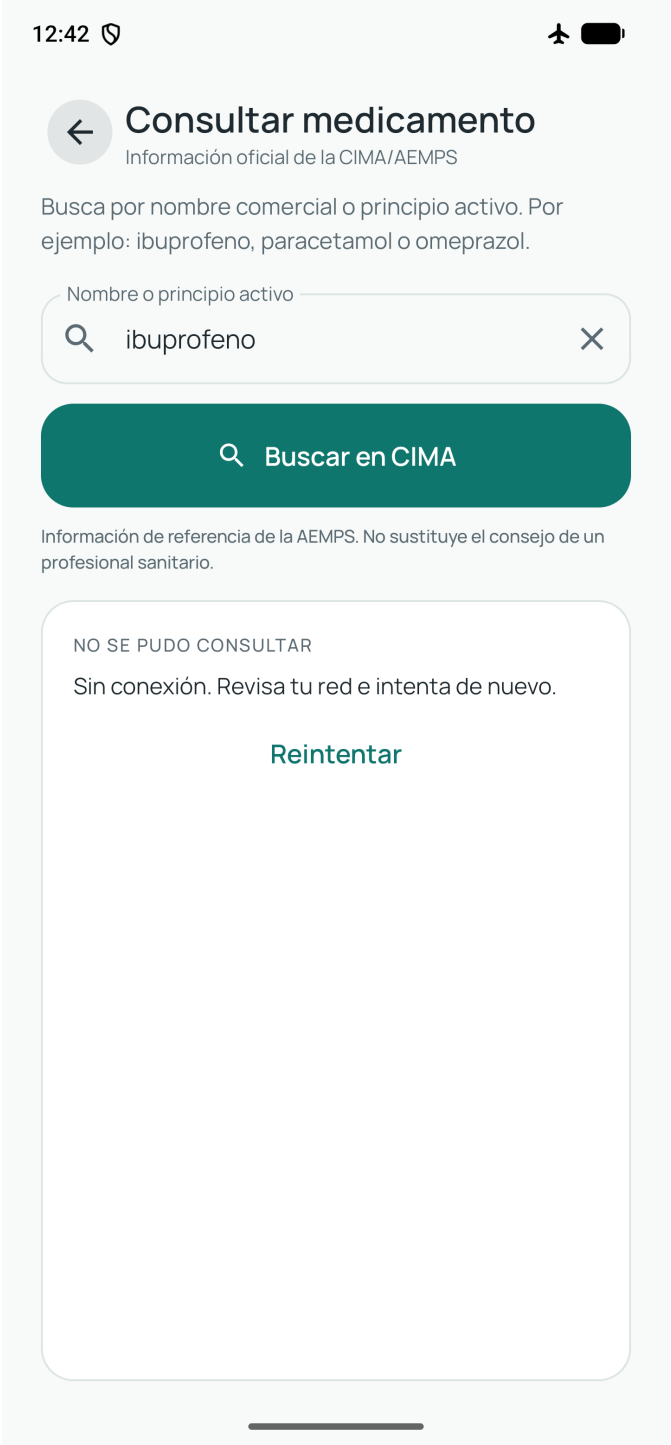
Evidencia 33. Resultados de CIMA en español.



Evidencia 34. Ficha detallada con documentos oficiales.



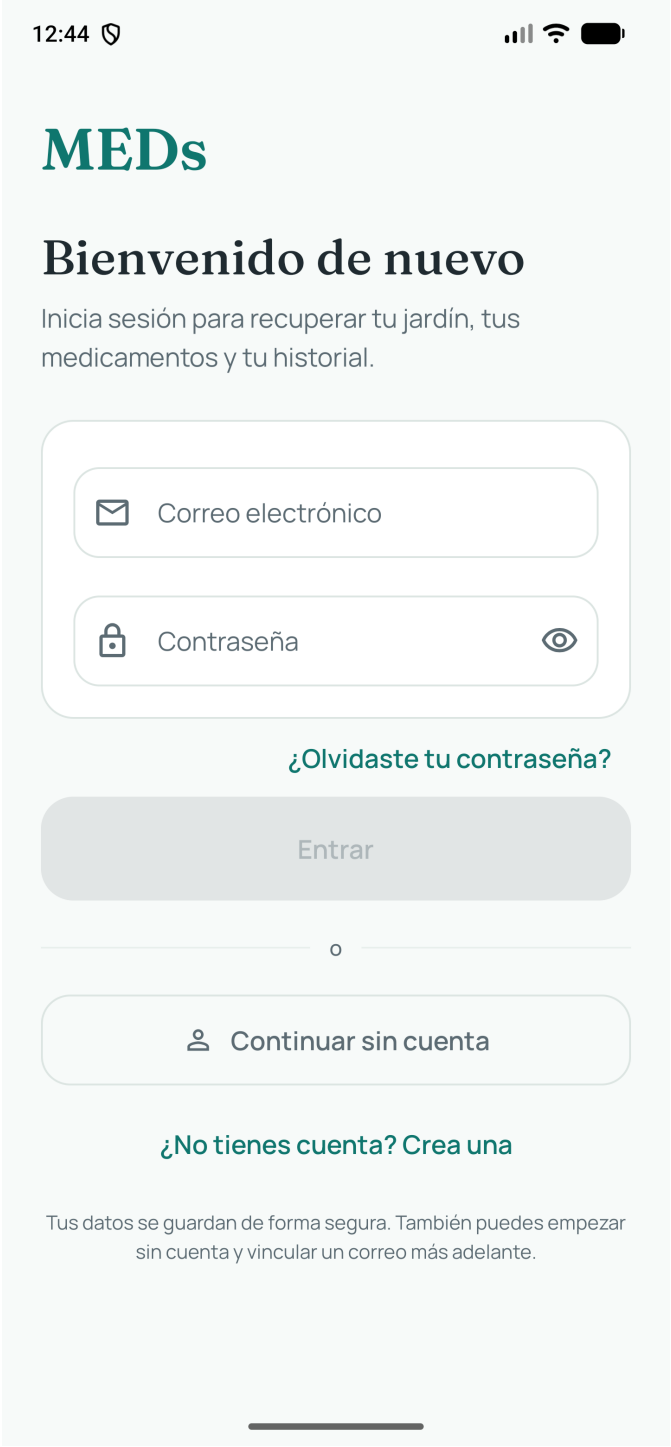
Evidencia 35. Estado sin resultados.



Evidencia 36. Error de conexión con reintento.



Evidencia 37. Biometría después del login.



Evidencia 38. Login sin biometría.

Anexos

A. Comandos útiles

```
# Compilar debug
.\gradlew.bat :app:assembleDebug

# Compilar release firmado
.\gradlew.bat :app:assembleRelease :app:bundleRelease

# Instalar el release
adb install -r dist\MEDs-release-26.1.1.apk

# Verificar firma
apksigner verify --print-certs dist\MEDs-release-26.1.1.apk

# Emuladores de Firebase (opcional)
npx --yes firebase-tools@latest emulators:start --only auth,firestore --project medsproto

# Desplegar Functions y reglas (requiere Blaze y firebase login)
firebase deploy --only functions,firestore:rules --project medsproto
```

B. Cuentas de prueba

Cuenta de paciente para la demostración: prueba.push.meds@example.com. La contraseña se comparte por un canal aparte del repositorio. También existe una cuenta de acompañante anónima en el emulador, vinculada a la paciente de prueba. Las cuentas de ejemplo deben eliminarse antes de publicar la app.

C. Glosario

Término	Significado
MVVM	Model-View-ViewModel: separa datos, estado e interfaz.
StateFlow	Flujo de estado observable e inmutable que consume Compose.
Repository	Capa que encapsula el acceso a datos y las reglas de negocio.
Batch	Escritura atómica de varios documentos en Firestore.
Retrofit	Cliente HTTP para consumir la API REST de CIMA.
CIMA	Centro de Información online de Medicamentos de la AEMPS.
Nudge	Recordatorio breve que el acompañante envía al paciente.
FCM	Firebase Cloud Messaging, servicio de push.
Keystore	Archivo de firma necesario para publicar actualizaciones.

D. Checklist final

- Release 26.1.1 firmado e instalado en el dispositivo físico.
- Reglas de Firestore publicadas y probadas.
- Functions y Blaze desplegados si se demuestra push cerrado.
- PDF, README y evidencias 1 a 38 actualizados.
- Tag v26.1.1 y Release de GitHub publicados con APK y AAB adjuntos.
- Cuenta de demostración y contraseña disponibles por canal aparte.
- Guion de 10 minutos ensayado.